Python Programming

Part 2 — Control Flow, Loops, Comprehensions & Practice Problems

Class Notes

Table of Contents

12. Conditional Statements — if, elif, else, Nested if, Ternary

Conditional statements let a program make decisions. Based on whether a condition is `True` or `False`, the program chooses which block of code to run. This is the foundation of decision-making logic in every program, including filters and branching rules used while processing data.

if — elif — else

Checks conditions one after another, top to bottom. The first condition that is True runs, and the rest are skipped.

```
marks = 72

if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
elif marks >= 60:
    print("Grade C")
else:
    print("Fail")
```

> **Output:**
> `Grade C`

Nested if-elif-else

An if statement placed inside another if statement. Used when a decision depends on more than one level of conditions.

```
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("Entry allowed")
    else:
        print("ID required")
else:
    print("Entry denied, underage")
```

> **Output:**
> `Entry allowed`

Ternary Operator (conditional expression)

A one-line short form of if-else, used when you just want to assign one of two values based on a condition.

Syntax: `value_if_true if condition else value_if_false`

```
age = 20
status = "Adult" if age >= 18 else "Minor"
print(status)
```

> **Output:**
> `Adult`

Comparison — which one to use

Form	Best for	Readability
if-elif-else	Multiple independent conditions in sequence	Good for 2-5 conditions
Nested if	A decision that depends on another decision	Can get messy if nested too deep
Ternary	Simple one-value assignment based on one condition	Very short, but hard to read if overused

Advantages

- Ternary reduces 4 lines of code to 1
- Nested if allows very precise, layered decisions

Limitations

- Deeply nested if statements become hard to read and debug
- Ternary should not be used for complex logic — it hurts readability

Applications

Where this is used:

- Labeling data based on rules, e.g. converting marks into grades, or age into an age-group category
- Validating data before it enters an analysis pipeline (e.g. checking if a value is within an expected range)
- Quick default-value assignment using ternary, e.g. `value = x if x is not None else 0`

EXAMPLE 1 — ODD OR EVEN

```
num = 15
if num % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Output:

```
Odd
```

EXAMPLE 2 — NESTED IF FOR A LOGIN-STYLE CHECK

```
username = "admin"
password = "1234"

if username == "admin":
    if password == "1234":
        print("Login Successful")
    else:
        print("Wrong Password")
else:
    print("User not found")
```

> **Output:**
> Login Successful

> 💼 **Common Interview Question:** Check whether a number is positive, negative, or zero.

EXAMPLE 3

```
num = -8

if num > 0:
    print("Positive")
elif num < 0:
    print("Negative")
else:
    print("Zero")
```

> **Output:**
> Negative

13. Loops — for loop, while loop, iter()

Loops let you repeat a block of code multiple times instead of writing it again and again. This is one of the most important concepts in programming, and in data science it is used everywhere — processing every row of a dataset, repeating a training step, or scanning through files.

for loop

Used when you know in advance how many times to repeat, or when you want to go through every item in a sequence (list, string, range, etc.) — this is called **definite iteration**.

```
for i in range(5):
    print(i)
```

> **Output:**
> `0 1 2 3 4`

while loop

Repeats a block of code as long as a condition remains True. Used when you don't know in advance exactly how many times the loop should run — this is called **indefinite iteration**.

```
count = 1
while count <= 5:
    print(count)
    count += 1
```

> **Output:**
> `1 2 3 4 5`

iter() and Iterators

An **iterable** is anything you can loop over (like a list or string). An **iterator** is the object that actually does the looping, one item at a time. The `iter()` function converts an iterable into an iterator, and `next()` gets the next value from it.

```
nums = [10, 20, 30]
it = iter(nums)

print(next(it))
print(next(it))
print(next(it))
```

Output:

```
10 20 30
```

Comparison — for vs while

Feature	for loop	while loop
Best used when	Number of repetitions is known / iterating a sequence	Repetitions depend on a condition, not a fixed count
Risk of infinite loop	Low	Higher, if condition never becomes False
Typical use	Looping through a list, string, or range	Waiting for user input, retry logic

Advantages

- for loop is clean and safe for sequences — no risk of infinite loop
- while loop is flexible for condition-based repetition

Limitations

- while loop can run forever if the condition is written incorrectly
- for loop is less natural when the stopping condition isn't about counting items

Applications

> **Where loops are used in Data Science:**
>
> - for loop: going through each row of a dataset, each file in a folder, each epoch in model training
> - while loop: retrying a failed API call until it succeeds, or running until a stopping condition (e.g. error below a threshold) is met
> - iter()/next(): used internally by Pandas and PyTorch DataLoader to feed data in batches

EXAMPLE 1 — FOR LOOP OVER A LIST

```
fruits = ["apple", "mango", "banana"]
for fruit in fruits:
    print(fruit)
```

> **Output:**
>
> ```
> apple mango banana
> ```

EXAMPLE 2 — WHILE LOOP, SUM OF DIGITS

> 💼 **Common Interview Question:** Find the sum of digits of a number.

```
num = 1234
total = 0

while num > 0:
    digit = num % 10
    total += digit
    num = num // 10

print(total)
```

> **Output:**
>
> ```
> 10
> ```

EXAMPLE 3 — FOR LOOP, FIBONACCI SERIES

> 💼 **Common Interview Question:** Print the first n Fibonacci numbers.

```
n = 7
a, b = 0, 1
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Output:

```
0 1 1 2 3 5 8
```

14. Loop with if-else Condition

Loops and conditions are combined constantly — you repeat an action, but only do something specific when a condition inside the loop is met. This pattern is at the heart of filtering and categorizing data.

Applications

Where this is used:

- Filtering rows of data that meet a condition (e.g. sales above a target)
- Categorizing every item in a dataset (pass/fail, high/medium/low)
- Counting how many items in a collection satisfy a rule

EXAMPLE 1 — FILTER EVEN NUMBERS FROM A LIST

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
for n in numbers:
    if n % 2 == 0:
        print(n, "is even")
```

Output:

```
2 is even 4 is even 6 is even 8 is even
```

EXAMPLE 2 — CATEGORIZE MARKS PASS/FAIL INSIDE A LOOP

```
marks_list = [45, 30, 78, 55, 20]
for marks in marks_list:
    if marks >= 40:
        print(marks, "-> Pass")
    else:
        print(marks, "-> Fail")
```

Output:

```
45 -> Pass 30 -> Fail 78 -> Pass 55 -> Pass 20 -> Fail
```

> 💼 **Common Interview Question:** FizzBuzz — print "Fizz" for multiples of 3, "Buzz" for multiples of 5, "FizzBuzz" for multiples of both, else the number.

EXAMPLE 3 — FIZZBUZZ

```
for i in range(1, 16):
    if i % 3 == 0 and i % 5 == 0:
        print("FizzBuzz")
    elif i % 3 == 0:
        print("Fizz")
    elif i % 5 == 0:
        print("Buzz")
    else:
        print(i)
```

> **Output:**
>
> ```
> 1 2 Fizz 4 Buzz Fizz 7 8 Fizz Buzz 11 Fizz 13 14 FizzBuzz
> ```

15. break, pass, continue

These three keywords control the flow inside a loop in different ways.

Keyword	What it does
break	Immediately stops and exits the loop completely
continue	Skips the current iteration and moves to the next one
pass	Does nothing — used as a placeholder so the code doesn't break

Advantages

- break saves time by stopping early once the answer is found
- continue keeps the loop clean by skipping unwanted cases
- pass lets you write the structure of a program before filling in logic

Limitations

- Overusing break/continue can make loop logic harder to follow
- pass is only a placeholder — forgetting to fill it in later leaves incomplete code

Applications

Where this is used:

- break: stop searching a dataset the moment a match is found (saves processing time on large data)
- continue: skip missing/invalid rows while processing a dataset instead of crashing
- pass: stub out a function while planning a data pipeline, to be completed later

EXAMPLE 1 — BREAK: STOP AT THE FIRST PRIME FACTOR

```
num = 30
for i in range(2, num):
    if num % i == 0:
        print("First factor found:", i)
        break
```

Output:

```
First factor found: 2
```

EXAMPLE 2 — CONTINUE: SKIP NEGATIVE NUMBERS

```
numbers = [5, -3, 8, -1, 9]
for n in numbers:
    if n < 0:
        continue
    print(n)
```

Output:

```
5 8 9
```

EXAMPLE 3 — PASS: PLACEHOLDER WHILE WRITING A FUNCTION

```
def clean_data(dataset):
    pass   # logic will be added later

print("Function defined, no error even though it's empty")
```

Output:

```
Function defined, no error even though it's empty
```

16. for-else and while-else

Loops in Python can have an `else` block too. It is a feature many other languages don't have. The `else` block runs only if the loop completes fully **without** hitting a `break`. If the loop is stopped early using `break`, the `else` block is skipped.

Applications

> **Where this is used:**
>
> - Searching for an item and printing "not found" only if the search completed without success
> - Validating that all items in a dataset passed a check, printing a message only if none failed

EXAMPLE 1 — FOR-ELSE, SEARCH IN A LIST

```
numbers = [4, 8, 15, 16]
search = 10

for n in numbers:
    if n == search:
        print("Found!")
        break
else:
    print("Not Found")
```

> **Output:**
>
> ```
> Not Found
> ```

EXAMPLE 2 — WHILE-ELSE, COUNTDOWN COMPLETES FULLY

```
count = 3
while count > 0:
    print(count)
    count -= 1
else:
    print("Countdown finished")
```

Output:

```
3 2 1 Countdown finished
```

💼 **Common Interview Question:** Check whether a number is prime using for-else.

EXAMPLE 3 — PRIME NUMBER CHECK

```
num = 17
for i in range(2, num):
    if num % i == 0:
        print(num, "is not prime")
        break
else:
    print(num, "is prime")
```

Output:

```
17 is prime
```

17. Nested Loops

A nested loop is a loop placed inside another loop. For every single run of the outer loop, the entire inner loop runs completely. Nested loops can mix types too — a `while` loop inside a `for` loop, or a `for` loop inside a `while` loop.

Comparison

Combination	Typical use case
for inside for	Fixed-size grids, tables, patterns (both dimensions known)
while inside for	Outer loop has a fixed count, inner loop depends on a changing condition
for inside while	Outer loop runs until a condition is met, inner loop processes a fixed sequence each time

Applications

> **Where this is used:**
>
> - Working with 2D data — tables, grids, matrices, images (rows and columns)
> - Printing patterns and multiplication tables
> - Comparing every item in one list with every item in another list

EXAMPLE 1 — FOR INSIDE FOR, MULTIPLICATION TABLE

> 💼 **Common Interview Question:** Print a multiplication table from 1 to 3.

```
for i in range(1, 4):
    for j in range(1, 4):
        print(i * j, end=" ")
    print()
```

Output:

```
1 2 3 2 4 6 3 6 9
```

EXAMPLE 2 — WHILE INSIDE FOR

```
for i in range(1, 4):
    j = 1
    while j <= i:
        print(j, end=" ")
        j += 1
    print()
```

Output:

```
1 1 2 1 2 3
```

EXAMPLE 3 — FOR INSIDE WHILE

```
i = 1
while i <= 3:
    for j in range(2):
        print(f"i={i}, j={j}")
    i += 1
```

Output:

```
i=1, j=0 i=1, j=1 i=2, j=0 i=2, j=1 i=3, j=0 i=3, j=1
```

18. List Comprehension

List comprehension is a short, one-line way to create a list, usually replacing a multi-line for loop. It is one of Python's most distinctive and most-loved features, and is used heavily in data science code because it is compact and fast.

Syntax: `[expression for item in iterable if condition]`

Comparison — list comprehension vs normal for loop

Aspect	List Comprehension	Normal for loop
Lines of code	1 line	3-4 lines
Speed	Usually faster	Slightly slower
Readability	Great for simple logic	Better for complex, multi-step logic

Advantages

- Very compact and readable for simple transformations
- Generally faster than an equivalent for loop

Limitations

- Becomes hard to read if the logic is complex or deeply nested
- Not ideal when multiple actions need to happen per item

Applications

Where this is used in Data Science:

- Quickly transforming a column of data, e.g. converting a list of prices to include tax
- Creating a filtered list of records without writing a full loop
- Cleaning text data in one line, e.g. removing spaces from every item in a list

EXAMPLE 1 — SQUARES OF NUMBERS

```
squares = [x**2 for x in range(1, 6)]
print(squares)
```

Output:

```
[1, 4, 9, 16, 25]
```

EXAMPLE 2 — FILTER EVEN NUMBERS

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8]
even = [n for n in numbers if n % 2 == 0]
print(even)
```

Output:

```
[2, 4, 6, 8]
```

💼 **Common Interview Question:** Flatten a nested list using list comprehension.

EXAMPLE 3 — FLATTEN A NESTED LIST

```
nested = [[1, 2], [3, 4], [5, 6]]
flat = [num for sublist in nested for num in sublist]
print(flat)
```

Output:

```
[1, 2, 3, 4, 5, 6]
```

19. Set Comprehension

Set comprehension works exactly like list comprehension, but it creates a set instead of a list — meaning the result automatically has no duplicate values.

Syntax: `{expression for item in iterable if condition}`

Comparison — list vs set comprehension

Aspect	List Comprehension	Set Comprehension
Result type	list — keeps duplicates and order	set — removes duplicates, unordered
Best used when	Order matters, duplicates are fine	You only need unique values

Applications

Where this is used:

- Getting unique values from a data column in one line, e.g. unique cities from a customer list
- Removing duplicate results while transforming data

EXAMPLE 1 — UNIQUE SQUARES (DUPLICATE INPUTS COLLAPSE AUTOMATICALLY)

```
nums = [1, -1, 2, -2, 3]
squares = {x**2 for x in nums}
print(squares)
```

Output:

```
{1, 4, 9}
```

💼 **Common Interview Question:** Find all unique vowels present in a string.

EXAMPLE 2 — UNIQUE VOWELS IN A STRING

```
text = "data science interview"
vowels = {ch for ch in text if ch in "aeiou"}
print(vowels)
```

Output:

```
{'a', 'e', 'i'}
```

EXAMPLE 3 — UNIQUE REMAINDERS

```
numbers = [10, 20, 23, 30, 33, 41]
remainders = {n % 10 for n in numbers}
print(remainders)
```

Output:

```
{0, 1, 3}
```

20. Practice Examples — List, Tuple, Set, Dict, String

This section brings together short practice problems on each data structure — the kind that are frequently asked in interviews to test how comfortable you are with Python fundamentals.

List

> 💼 **Common Interview Question:** Reverse a list without using the built-in reverse() function.

EXAMPLE 1

```
nums = [10, 20, 30, 40]
reversed_list = nums[::-1]
print(reversed_list)
```

> **Output:**
> `[40, 30, 20, 10]`

EXAMPLE 2 — REMOVE DUPLICATES FROM A LIST

```
nums = [1, 2, 2, 3, 4, 4, 5]
unique_nums = list(set(nums))
print(unique_nums)
```

> **Output:**
> `[1, 2, 3, 4, 5]`

Tuple

EXAMPLE 1 — SWAP TWO VARIABLES USING A TUPLE

```
a, b = 5, 10
a, b = b, a
print(a, b)
```

Output:

```
10 5
```

EXAMPLE 2 — COUNT OCCURRENCES IN A TUPLE

```
marks = (78, 90, 78, 65, 78)
print(marks.count(78))
```

Output:

```
3
```

Set

💼 **Common Interview Question:** Find common elements between two lists.

EXAMPLE 1

```
list1 = [1, 2, 3, 4]
list2 = [3, 4, 5, 6]
common = set(list1) & set(list2)
print(common)
```

Output:

```
{3, 4}
```

EXAMPLE 2 — ELEMENTS ONLY IN ONE SET (DIFFERENCE)

```
a = {1, 2, 3, 4}
b = {3, 4, 5}
print(a - b)
```

Output:

```
{1, 2}
```

Dictionary

> 💼 **Common Interview Question:** Count the frequency of each character in a string.

EXAMPLE 1

```
text = "data"
freq = {}
for ch in text:
    freq[ch] = freq.get(ch, 0) + 1
print(freq)
```

> **Output:**
> `{'d': 1, 'a': 2, 't': 1}`

EXAMPLE 2 — MERGE TWO DICTIONARIES

```
dict1 = {"a": 1, "b": 2}
dict2 = {"c": 3, "d": 4}
merged = {**dict1, **dict2}
print(merged)
```

> **Output:**
> `{'a': 1, 'b': 2, 'c': 3, 'd': 4}`

String

> 💼 **Common Interview Question:** Check if a string is a palindrome (reads the same forwards and backwards).

EXAMPLE 1

```
text = "madam"
print(text == text[::-1])
```

> **Output:**
> `True`

EXAMPLE 2 — COUNT VOWELS IN A STRING

```
text = "data science"
count = 0
for ch in text:
    if ch in "aeiou":
        count += 1
print(count)
```

Output:

4

21. Pattern Making using Loops

Pattern printing programs use nested loops — the outer loop controls the number of rows, and the inner loop controls what gets printed in each row. These are extremely common in college exams and beginner-level coding interviews because they test your control over nested loops.

Applications

Why this is asked:

- Tests understanding of nested loops and loop counters
- Builds the same "row by row, column by column" thinking used later for image/matrix processing in data science

EXAMPLE 1 — RIGHT-ANGLED TRIANGLE OF STARS

```
rows = 5
for i in range(1, rows + 1):
    print("*" * i)
```

Output:

```
* ** *** **** *****
```

💼 **Common Interview Question:** Print a number pattern like a triangle of increasing numbers.

EXAMPLE 2 — NUMBER TRIANGLE

```
rows = 5
for i in range(1, rows + 1):
    for j in range(1, i + 1):
        print(j, end=" ")
    print()
```

Output:

```
1 1 2 1 2 3 1 2 3 4 1 2 3 4 5
```

EXAMPLE 3 — PYRAMID PATTERN

```
rows = 4
for i in range(1, rows + 1):
    print(" " * (rows - i) + "*" * (2 * i - 1))
```

Output:

```
* *** ***** *******
```

Tip: Whenever a pattern problem confuses you, first figure out the pattern for the outer loop (row count), then focus separately on what the inner loop should print for each row.